EEE 475/575 Medical Image Reconstruction & Processing
Homework #2
31 March 2026, Tuesday at 23:59

GUIDELINES FOR HOMEWORK SUBMISSION

- Upload your solutions to Moodle as **two separate files**:
  1) **PDF Report File:** The report should be typeset (i.e., no handwriting) and should properly present your results. In addition, the report should display the relevant sections of your code at the end of each subpart. No partial credits will be given to answers with missing codes or unjustified answers.
     At the beginning of your report, provide a section titled "GenAI Usage Disclosure" and clearly explain whether you have used any genAI tools. Include the name/version of the tool and a brief description of how it was used (see the "GenAI Policy" included below).
  2) **Code File:** You can use any programming platform of your choice. If you do not upload your code file, your homework will not be evaluated (i.e., 0 pts).
- Submission system will remain open for 1 day after the deadline:
  - No points will be lost if you submit your assignment within 12 hours past the deadline.
  - There will be a 50% penalty if you submit after 12 hours but within 24 hours past the deadline.
  - No submissions beyond 24 hours past the deadline.

GenAI Policy: Generative AI (genAI) tools (e.g., large language models, coding assistants) may be used for homework assignments and project to assist with coding and to improve writing. Any use of genAI must be clearly disclosed, including the name/version of the tool and a brief description of how it was used. Upon request, the student should be ready to provide prompts, outputs, and execution logs pertaining to their genAI use for all assignments. Students remain fully responsible for all submitted material and must be able to explain and justify their code, results, and written content. Reliance on genAI does not transfer responsibility for correctness or originality. Failure to disclose AI usage or inappropriate reliance on AI-generated content will be treated as a violation of academic integrity.

HOMEWORK #2: PREPARATION

From the Moodle page of the class, download the following files:

- **spiral.mat:** Complex MRI data in k-space for an "interleaved" spiral k-space acquisition. Each interleaf starts from the center of k-space and spirals out. There are 6 interleaves, with 2048 data points per interleaf.
  - kdata: complex k-space data
  - ktraj: k-space trajectory given as ktraj = kx + i*ky as a complex variable. Here, ktraj is scaled to ±kmax = ±0.5, so that $K_{extent} = 1$. Consequently, the size of each pixel is $\Delta x = 1/K_{extent} = 1$. So, this normalization makes the pixel size equal to 1.
- **shepplogan_radial_data.mat:** Complex MRI data in k-space for a radial k-space acquisition. Each radial line starts from the center of k-space and extends out at an angle. There are 300 radial lines, with 129 data points per radial line. kdata and ktraj are complex k-space data and k-space trajectory, as before.
- **voronoidens.m:** Calculates the area associated with each k-space point on a trajectory. Type "help voronoidens" in MATLAB to see how this function works.
- **gridkb.m:** Code for "gridding reconstruction" that was briefly explained in class (see the first page of lecture notes). This code resamples the k-space data on a Cartesian grid following convolution with a chosen kernel, and then applies deapodization correction in image domain. Type "help gridkb" in MATLAB to see how this function works.

Important Note #1: The reconstructions in this homework may not preserve the absolute scale of the image (i.e., the image may be multiplied by some arbitrary constant). Not knowing the scaling makes it difficult to perform qualitative and quantitative comparison among different methods. For simplicity, normalize each image by its maximum value. In MATLAB, this can be done as follows:

```
>> ima = abs(ima)/max(abs(ima(:)));
```

where `ima` is the reconstructed image. Only after this normalization these step, you can display the image and/or perform IQA analysis. Note that this normalization is prone to errors, as we are using a single pixel to determine the scale of an entire image. If a single pixel has unexpectedly high/low intensity, it will dominate IQA metrics.

Important Note #2: For all error images (i.e., the magnitude of the difference between the reconstructed image and the reference image), use a fixed *imshow* grayscale window of [0 0.2]. Using a fixed window enables us to visually compare error images across different methods.

Homework #2

PART I – GRIDDING RECONSTRUCTION (52 pts)

1.1) Load spiral.mat. Display the 2D k-space trajectory, where the x-axis is kx (real part of ktraj) and the y-axis is ky (imaginary part of ktraj). In MATLAB, this can be done as follows:

```
>> plot(ktraj);
```

Clearly label both axes of your plot. Here, *plot* function plots the columns of ktraj. Since ktraj is complex-valued, *plot(ktraj)* is equivalent to *plot(real(ktraj),imag(ktraj))*. So, MATLAB treats the real part of each column as the x-axis and the imaginary part of each column as the y-axis. What you see is a so-called "interleaved spiral trajectory", which has a total of 6 interleaves (each shown with a different color in this plot). Each interleaf is acquired during a separate repetition time (TR) after the excitation of the 2D slice, and has 2048 data points. Hence, both ktraj and kdata have size 2048×6.

1.2) Calculating Density Compensation Filter: Next, use the voronoidens.m function to estimate the density on the area associated with each k-space point on the trajectory:

```
>> area = voronoidens(ktraj);
```

Note that the area directly gives the density compensation factor that we want to use. Plot the computed area:

```
>> plot(area(:));
```

Clearly label both axes. Also plot a zoomed-in version that shows the reasonable values in this plot. How many NaN values does the computed area have? Why are there NaN values? Why are some of the area values unreasonably large? Briefly explain.

1.3) Correcting the Density Compensation Filter: Now, find a way to assign reasonable values to the computed areas. As we discussed in class, there are many ways to do this, so there is no single correct answer. For example, you could do a simple thresholding. Whatever method you use, make sure that there are no NaN values remaining. Plot the resulting corrected area values.

1.4) Direct Summation: The direct summation approach for reconstructing non-Cartesian data can be expressed as follows for the 1D case (as explained in class):

$$m[n] = \sum_{i=1}^{K} d_i M(k_i)\, e^{j2\pi k_i \Delta x\,(n-\tau)}, \quad n = 0, \ldots N-1 \qquad \text{(Eq.1)}$$

where $M(k_i)$ is the k-space data at k-space position k_i, d_i are density compensation factors (i.e., area computed in Question 1.3) at k-space position k_i, and Δx is the size of a pixel. As explained above, $\Delta x = 1$ in our case. In addition, τ indicates the center of the field-of-view (FOV) relative to the image, and hence is equal to $\tau = N/2$.

This technique is slow, but is used as the "gold standard" method in some papers. First, write the

direct summation equation for the 2D case. Then, implement the direct summation technique. Reconstruct an image with a size of N×N=128×128. This reconstruction approach is slow. In order to avoid unnecessarily lengthy computation times, pay attention to use vector arithmetic instead of *for-loop* iterations whenever possible.

Display the resulting image. Comment on the problems that you see in the image. We will use this image as the *reference image* for the rest of Part I questions.

Plot the central horizontal cross-section of this image:

```
>> plot(abs(ima_direct(end/2,:));
```

Also, plot the central vertical cross-section of this image:

```
>> plot(abs(ima_direct(:,end/2));
```

Lastly, state the computation time needed for the direct summation technique (in MATLAB, this can be done via the built-in *cputime* function). When computing the CPU time, only consider the part of your code where you are reconstructing the image (i.e., exclude the parts where you are displaying the results, etc.).

1.5) No Density Compensation: Repeat Part (1.4) without density compensation, i.e., remove d_i from the equation. Display the resulting image. Display the error image. Plot the central horizontal and vertical cross-sections of this image. In these 1D plots, overlay the horizontal and vertical cross-sections of the reference image for visual comparison. Compute PSNR, SSIM, and CPU time. Briefly comment on the results.

1.6) Double FOV: When reconstruction non-Cartesian data, we are free to choose the FOV as we wish. Repeat Part (1.4) to reconstruct an image with the twice the FOV but with same pixel size. To do this, keep $\Delta x = 1$ but set N×N=256×256. First, display the full 256×256 image that result from direct summation. Briefly comment on this image. Next, crop the outer parts of this image and display the central 128×128 image. Also, display the error image and the cross-sections for this central part (with cross-sections of the reference image overlayed). Compute PSNR, SSIM, and CPU time. Briefly comment on the results. In your opinion, was the increased CPU time worth it? Why or why not?

1.7) Half Pixel Size: When reconstruction non-Cartesian data, we are free to choose the pixel size as we wish. Repeat Part (1.4) to reconstruct an image with the half the pixel size but the same FOV. To do this, set $\Delta x = 0.5$ and N×N=256×256. Display the full 256×256 image. To compare this image with the reference image, we need to zero pad the reference image in Fourier domain (refer back to Part (1.3) of HW#1). Perform this operation for the reference image, and use the resulting image as the new reference (for this question only). Next, display the error image and the cross-sections for the half-pixel-size image (with cross-sections of the new reference image overlayed). Compute PSNR, SSIM, and CPU time. Briefly comment on the results. In your opinion, was the increased CPU time worth it? Why or why not?

1.8) Double Pixel Size: Repeat Part (1.4) to reconstruct an image with the double the pixel size but the same FOV. To do this, set $\Delta x = 2$ and N×N=64×64. Display the resulting 64×64 image. To compare this image with the reference image, we need to zero pad it. Perform this operation and display the resulting image. Next, display the error image and the cross-sections for the double-pixel-size image (with cross-sections of the new reference image overlayed). Compute PSNR, SSIM, and CPU time. Briefly comment on the results.

1.9) 1X Gridding Reconstruction: In class, we have briefly discussed the gridding reconstruction that is based on k-space convolution and resampling. The provided MATLAB gridkb.m performs this reconstruction. Reconstruct an image with a 128×128 size, an oversampling factor of osf=1, a kernel width of wg=2 as follows:

```
>> ima = gridkb(kdata,ktraj,area,128,1,2,'image');
```

The oversampling factor (osf) refers to the grid size in k-space with respect to the targeted FOV size. In this case, the k-space data will be resampled on a 128×128 Cartesian grid that extends from ±kmax = ±0.5. A convolution kernel of width that spans 2 k-space grids is utilized for convolution.

Display the resulting image and the error image. Plot the cross-sections of this image (with cross-sections of the reference image overlayed). Compute PSNR, SSIM, and CPU time. Briefly comment on the results and the problems that you see in the image.

1.10) 2X Gridding Reconstruction: Repeat Part (1.9) for osf = 2, for which the k-space data will be resampled on a 256×256 Cartesian grid that extends from ±kmax = ±0.5. As a result, k-space grid distances will be halved, so that the reconstructed image will have twice the FOV. For this case, the kernel width needs to be set to wg = 4, so that the convolution kernel covers the same effective radius in this 2X grid as it did for the 1X grid.

```
>> ima = gridkb(kdata,ktraj,area,128,2,4,'image');
```

First, display the resulting 256×256 image. Next, crop the outer parts of this image and display the central 128×128 image. Also, display the error image and the cross-sections of the central part (with cross-sections of the reference image overlayed). Compute PSNR, SSIM, and CPU time. Briefly comment on the results.

Important Note: *For Question 1.11-1.12 below, the resulting images may be flipped/transposed due to flipped/swapped k-space axes from meshgrid. So, flip/transpose them back before you compare with the reference image.*

1.11) Scattered Interpolation with 1X Grid: As an alternative approach for gridding reconstruction, for every grid point, we can interpolate between nearest data points. Since this approach does not use all of the data points, it is not preferred in MRI. This approach corresponds to *scatteredInterpolant* function of MATLAB. Type "help scatteredInterpolant" to see how this function works. Using the *meshgrid* function of MATLAB, define a 128×128 Cartesian k-space grid that extends from ±kmax = ±0.5. To interpolate the data onto this Cartesian grid, use *scatteredInterpolant* with interpolation method set to 'linear' and extrapolation method set to

'none'. Note that the Cartesian grid extends outside the circular region of support (ROS) of the k-space trajectory. Since we set extrapolation method to 'none', those grid points will be assigned NaN values. Set those NaN values to zero. Finally, take inverse 2DFT to find the image. Display the results as before (image, error image, overlayed cross-sections). Compute PSNR, SSIM, and CPU time. Briefly comment on the differences between this image and the result from Part (1.10).

1.12) Scattered Interpolation with 2X Grid: Repeat Part (1.11) using a 2X grid, i.e., 256×256 Cartesian k-space grid that extends from ±kmax = ±0.5. Display the results as before (full image, central part of the image, error image, overlayed cross-sections). Compute PSNR, SSIM, and CPU time. Briefly comment on the differences between this image and the result from Part (1.10).

PART II – GRIDDING VS. BACKPROJECTION FOR PROJECTION DATA (30 pts)

2.1) Generating Projections: In MATLAB, the built-in *radon* function generates the projections for a given input (i.e., computes its Radon Transform). First, generate a 256×256 phantom image and then its projections for angles from 0 degree to 179 degree:

```
>> P = phantom('Modified Shepp-Logan',256);
>> proj = radon(P,[0:179]);
```

Display the phantom and its central horizontal and vertical cross-sections. We will use the phantom itself, P, as the ***reference image*** **for both Part II and Part III questions**. Display the sinogram (i.e., proj). Here, proj matrix has size 367x180, where 180 is the number of projections. One axis is l and the other axis is θ. Clearly label both axes of the sinogram.

2.2) Filtered Backprojection: Using *iradon* function of MATLAB, reconstruct the image from its projections (i.e., compute the inverse Radon Transform). Type "help iradon" to see how this function works. Use the default filter in *iradon* (called Ram-Lak in Matlab), which is the ramp filter that we have covered in class. Note that *iradon* will yield an image of size 258×258. Use the central 256×256 of this image as your result. Display the results as before (image, error image, overlayed cross-sections). Compute PSNR, SSIM, and CPU time. Briefly comment on the result.

2.3) Naïve Backprojection: Repeat Question 2.2 for the case of no filter (i.e., 'none' option for the filter in iradon). This corresponds to naïve backprojection. Display the results as before (image, error image, overlayed cross-sections). Compute PSNR, SSIM, and CPU time. Briefly comment results.

2.4) Generating k-space Data from Projections: Now, using the Projection-Slice Theorem, compute the k-space data (a 367×180 matrix) and the corresponding k-space trajectory (a 367×180 matrix) for this case. For the k-space trajectory, we have

```
>> ktraj = kx + i*ky;
```

Here, set the maximum k-space radius as 0.5, so that your data will be compatible with the *gridkb* function. Plot the k-space trajectory. Display the magnitude of the k-space data (i.e., the **1D** Fourier transform of the sinogram along the l-axis) as an image (using *log* format as explained

in HW#1). Here, the k-space data is a 367×180 image, where one axis is k_r (or ρ) and the other axis is θ. Clearly label both axes.

2.5) Calculating Density Compensation Filter: Using the geometric approach for the radial scanning case (as explained in lecture notes), compute the area associated with each k-space point. This will also be a 367×180 matrix. Plot the resulting area values for the first angle only.

Side Note: Voronoi method expects unique positions for each data point, but the k-space trajectory passes through the k-space center for all projections. Hence, it will give an error for this case.

Important Note: *For Questions 2.6-2.8 below, the resulting images may be flipped/transposed due to flipped k-space axes. So, flip/transpose them back to compare with the reference image.*

2.6) Direct Summation: Compute the direct summation image using the k-space data, trajectory, and the density compensation filter that you computed. Display the results as before (image, error image, overlayed cross-sections). Compute PSNR, SSIM, and CPU time. Briefly comment on the results.

2.7) 2X Gridding Reconstruction: Perform 2X gridding reconstruction with a kernel width of wg=4, like you have done in Part (1.10). Display the results as before (full image, central part of the image, error image, overlayed cross-sections). Compute PSNR, SSIM, and CPU Time. Briefly comment on the results.

PART III – GRIDDING FOR RADIAL DATA (18 pts)

3.1) Load shepplogan_radial_data.mat. Here, both the trajectory and k-space data are 129×300 matrices, where 129 is the number of samples for each line, and 300 is the number of radial lines. Display the 2D k-space trajectory. Clearly label both axes.

3.2) Calculating Density Compensation Filter: This data is slightly different than the k-space data of a projection. Here, the radial lines start at the center of k-space and extend out in one direction only. Using the geometric approach, compute the area associated with each k-space point. This will also be a 129×300 matrix. Plot the resulting area values for one radial line only.

Important Note: *For Questions 3.3-3.5 below, the resulting images may be flipped/transposed due to flipped k-space axes. So, flip/transpose them back to compare to the reference image.*

3.3) Direct Summation: Compute the direct summation image using the k-space data, trajectory, and the density compensation filter that you computed. Display the results as before (image, error image, overlayed cross-sections). Compute PSNR, SSIM, and CPU time. Briefly comment on the results.

3.4) 2X Gridding Reconstruction: Perform 2X gridding reconstruction. Display the results as before (full image, central part of the image, error image, overlayed cross-sections). Compute PSNR, SSIM, and CPU Time. Briefly comment on the results.